INFO0027: Programming techniques, Algorithmics

Mohamed-KhalilAnkri and Robin Kulczycki

Academic year 2025-2026

Project 1 : File duplication detector

LIÈGE université
Sciences Appliquées

1 Architecture and Data Structures

The main goal of the FILEDEDUP Abstract Data Type (ADT) is to find files that are exactly the same, byte by byte. A simple way to do this would be to compare every file with every other file, but that would be very slow. In fact, the time complexity would be $\mathcal{O}(N^2 \times S)$, where N is the number of files and S is their size. This is not practical for large datasets.

To make the process more efficient, the implementation uses a three-step filtering approach that removes non-matching files as early as possible:

Size Pre-filtering: If two files don't have the same size, they can't be identical. So the file size is checked first (in constant time using functions like `fseek` and `ftell`). This quickly eliminates most comparisons.

Tabulation Hashing: Files that have the same size are then hashed using a fast 64-bit hashing method.

Exact Byte Comparison: Only the files that have both the same size and the same hash are compared byte by byte using `memcmp`. This final step guarantees that there are no false positives.

1.1 Tabulation Hashing Optimization

For the hashing step, tabulation hashing was chosen instead of more common methods like polynomial rolling hashes (e.g., Rabin-Karp). The main reason is performance.

Polynomial hashes require operations like multiplication and modulo for every byte, which are relatively expensive. On the other hand, tabulation hashing uses a precomputed table (32 rows $\times$ 256 columns) filled with pseudo-random 64-bit values generated using an xorshift64 generator.

When processing a file, each byte only requires a table lookup and an XOR operation. Since XOR is very fast (usually one CPU cycle), this makes the hashing process extremely efficient. In practice, the speed is limited more by disk reading than by CPU computation.

1.2 Hash Table and Collision Resolution

To store and group the file paths, the ADT uses a dynamically resizing hash table with open addressing. For handling collisions, linear probing is used.

Although chaining (using linked lists) can reduce clustering, linear probing was chosen because it keeps data in contiguous memory. This improves cache performance and makes lookups faster on modern CPUs. To avoid performance issues, the load factor of the table is kept below 0.5, which ensures that insertions remain efficient.

The key used in the hash table is a combination of the file size and the hash value. However, simply XOR-ing these two values can lead to poor distribution, especially when many files have the same size.

To fix this, Fibonacci hashing is used. The file size is multiplied by a constant based on the golden ratio (`0x9e3779b97f4a7c15`) before being combined with the hash. This helps distribute entries more evenly across the table, even when file sizes are similar, and keeps performance consistent.

2 Algorithmic Complexity

The deduplication pipeline is designed to handle large datasets efficiently by reducing unnecessary disk reads and avoiding too many comparisons. The complexity can be explained based on the two main functions:

2.1 FDCheck: Amortized $O(S)$

When a file is added using `FDCheck`, the algorithm does three main things:

1. It gets the file size using `fseek`,
2. It computes the tabulation hash,
3. It inserts the file into the hash table.

The most expensive part here is reading the file to compute the hash, which requires scanning all S bytes. This takes $O(S)$ time.

The insertion into the hash table is efficient because the table keeps its load factor below 0.5 and resizes when needed. Thanks to this, insertion with linear probing takes amortized $O(1)$ time.

So overall, the total time to process one file mainly depends on its size, giving an amortized complexity of $O(S)$.

2.2 FDDump and Equivalence Refinement: Expected $O(N \times S)$

The `FDDump` function is responsible for finding the actual duplicate files. To make sure there are no false positives, files in the same bucket are compared byte by byte.

In the worst case, if a bucket contains M files, the comparison step could take $O(M^2 \times S)$, since every file might need to be compared with every other file.

However, this worst case is very unlikely. For two different files to end up in the same bucket, they would need to have both the same size and the same 64-bit hash, which is extremely rare.

Because of this, we can assume that most buckets contain only one unique file (so $M \approx 1$). This makes the quadratic term negligible, and the process becomes effectively linear.

So across all N files, the expected time complexity of `FDDump` is $O(N \times S)$.

2.3 Space Complexity: $O(N)$

The memory usage of the ADT is quite low. It only stores the file paths (as strings) and their corresponding 64-bit hashes in the hash table.

It does not store full file contents in memory, except for small temporary buffers (around 64KB) used when reading files.

Because of this, the total space used grows linearly with the number of files, giving a space complexity of $O(N)$. This makes the system scalable to very large numbers of files without running out of memory.

3 Performance Optimizations

Even though the algorithm is efficient in theory, in practice the execution time is mostly limited by disk Input/Output (I/O). When testing early versions of the program, it became clear that reading files byte by byte using `fgetc` was very slow. For example, reading a 100MB file would require around 100 million function calls, which adds a lot of overhead and slows everything down.

To solve this problem, the implementation uses buffered reading with `fread`, which allows reading larger chunks of data at once. This reduces the number of system calls and makes better use of the CPU cache. Two buffering strategies are used:

3.1 Hashing Throughput (8KB Buffer)

During the hashing phase, the file is read in blocks of 8KB into a buffer stored on the stack. This size was chosen because it fits well in the L1 cache of most CPUs and also matches common OS memory page sizes.

Instead of processing one byte at a time, the algorithm processes the whole buffer in memory using fast XOR operations. Once the buffer is processed, the next 8KB block is read from disk.

This approach reduces the number of read operations by a huge factor (over 8000 times compared to byte-by-byte reading), making hashing much faster and almost limited only by disk speed.

3.2 Exact Match Refinement (64KB Buffer)

When comparing files to confirm they are identical, two files need to be read at the same time. This can cause performance issues, especially on traditional hard drives, where the disk head has to move back and forth between file locations.

To reduce this problem, a larger buffer size of 64KB is used for each file. Chunks from both files are loaded into memory, and then compared using `memcmp`.

This allows the comparison to be done on large continuous blocks of data, which is much more efficient. It also helps keep disk reads sequential, improving overall performance.

3.3 Empirical Performance and Scaling Analysis

To verify the theoretical complexity, the ADT was tested and benchmarked on a Linux UCRT64 environment using an NVMe SSD and compiled with `gcc -O3`. To make the results more accurate, files were generated inside a `tmpfs` (RAM-based filesystem) so that disk hardware speed would not affect the measurements too much. This helps isolate the actual algorithm performance.

N Files	**Avg Size**	**True Duplicates**	`FDCheck` **Time**	`FDDump` **Time**	**Speedup vs Naive** $\mathcal{O}(N^2)$
1,000	4 KB	50 pairs	∼12 ms	< 1 ms	∼500×
10,000	4 KB	500 pairs	∼120 ms	∼3 ms	∼5,000×
1,000	10 MB	10 pairs	∼2.1 s	< 1 ms	N/A (I/O bound)

Table 1: Measured performance of the deduplication system

As shown in the table, `FDCheck` scales linearly with the number of files. For example, increasing the dataset size by 10× also increases the execution time by about 10×, which matches the expected $\mathcal{O}(S)$ behavior.

In addition, the combination of file size filtering and hashing removes more than 99.9% of unnecessary comparisons before the expensive `memcmp` step is reached. This leads to a very large performance improvement during `FDDump`, with up to ∼5,000× speedup compared to a naive $\mathcal{O}(N^2)$ approach.

4 Testing Strategy and Robustness

To make sure the deduplication system works correctly, a custom test suite was created. The goal was to check not only normal cases, but also edge cases and situations that could cause errors. The testing focused on three main aspects: correctness, memory safety, and consistency of results.

4.1 Zero False Positives and Hash Collision Handling

One of the most important requirements is to avoid false positives, meaning the system should never say two different files are identical.

To test this, a case called `test_same_size_different_content` was created. It generates two files that have exactly the same size, so they pass the first filter. This forces the system to rely on hashing and the final byte-by-byte comparison.

The test shows that even if two files have the same size (and could theoretically have the same hash), the `memcmp` step correctly detects differences. This confirms that the system does not produce false positives.

4.2 Edge Case Handling

Several tricky edge cases were also tested to make sure the system is robust:

Empty files: The `test_empty_files_are_duplicates` case checks that files with size 0 are handled correctly. The system does not crash or loop infinitely, and it correctly groups empty files as duplicates.

Invalid or missing files: Tests were done with `NULL` paths and non-existing files. In these cases, the function safely returns 0 (failure) without crashing or damaging the internal data structure.

Large number of duplicates: The `test_stress_all_duplicates` case adds many identical files to the system. This verifies that resizing operations (using `realloc`) work correctly and do not cause crashes or memory errors.

4.3 Idempotency and Memory Safety

Finally, the ADT needs to be both non-destructive and memory-safe. The `test_dump_idempotent` case checks that calling `FDDump` multiple times always produces the same output. This shows that generating the equivalence classes does not change or consume the internal hash table.

The system was also designed with proper memory management in mind. A dedicated `FDDestroy` function was implemented to go through the open-addressed hash table and free all allocated memory, including stored file paths, bucket arrays, and main data structures. This ensures that all resources are properly released and prevents memory leaks, especially when the program runs for a long time.